

Image Analysis — Assignment 2

(Solutions and Report)

Student:

Programme: MLSC

September 2025

Contents

1	Filtering	2
2	Interpolation	2
3	Classification using Nearest Neighbour and Bayes theorem	6
4	Image Classification	8
5	Line Classification	9
6	Character Classification	10
7	Feature Extraction for OCR	11

1 Filtering

By analyzing the effects of the different convolution kernels, the following correspondences can be established:

- $\mathbf{A} \rightarrow f_6$

Kernel f_6 contains only negative weights, which inverts the local average values and produces a global contrast inversion (black becomes white and vice versa). Image A shows this inverted effect.

- $\mathbf{B} \rightarrow f_2$

Kernel f_2 acts as a smoothing operator: the center pixel is excluded while the surrounding pixels are averaged. This reduces image sharpness and produces a slightly blurred, softened version of the original image. Image B clearly matches this effect.

- $\mathbf{C} \rightarrow f_1$

Kernel f_1 can be interpreted as vertical smoothing combined with horizontal differencing. It emphasizes horizontal intensity variations, resulting in paired edges and a relief-like structure. Image C shows this characteristic.

- $\mathbf{D} \rightarrow f_3$

Kernel f_3 emphasizes diagonal differences (positive in the NE direction, negative in the SW direction), creating a diagonal embossing effect. Image D corresponds to this transformation.

- $\mathbf{E} \rightarrow f_5$

Kernel f_5 is essentially a shift operator, taking the value of a neighboring pixel. As a result, the image is displaced horizontally. Image E looks almost identical to the original but shifted. Moreover, because zero-padding is applied at the border, the left side of the shifted image displays a visible black strip, which further confirms the displacement property of this kernel.

The unused kernel is f_4 , which is a sharpening operator (large positive center surrounded by negative values). It enhances edges and details, but no such effect is observed in the provided images.

2 Interpolation

(a) Linear interpolation

Definition and interpretation.

Linear interpolation is the simplest form of interpolation, where the interpolating function is constructed as a continuous, piecewise linear function that exactly passes through all given sample points. Intuitively, it corresponds to connecting each pair of consecutive samples by

straight line segments. This guarantees that the interpolant reproduces the original data at the integer positions while providing a continuous transition in between.

Mathematically, the linear interpolant can be expressed as a weighted superposition of shifted “hat” basis functions,

$$F_{\text{lin}}(x) = \sum_{i=1}^7 g_{\text{lin}}(x - i) f(i), \quad g_{\text{lin}}(x) = \max(1 - |x|, 0),$$

where $g_{\text{lin}}(x)$ is the triangular basis function with compact support $[-1, 1]$.

Explicit piecewise expression.

For the given samples

$$f = [1, 4, 6, 8, 7, 5, 3],$$

the interpolant on each interval $[i, i + 1]$ is the straight line between $(i, f(i))$ and $(i + 1, f(i + 1))$:

$$F_{\text{lin}}(x) = \begin{cases} 1 + 3(x - 1), & x \in [1, 2], \\ 4 + 2(x - 2), & x \in [2, 3], \\ 6 + 2(x - 3), & x \in [3, 4], \\ 12 - x, & x \in [4, 5], \\ 17 - 2x, & x \in [5, 6], \\ 17 - 2x, & x \in [6, 7]. \end{cases}$$

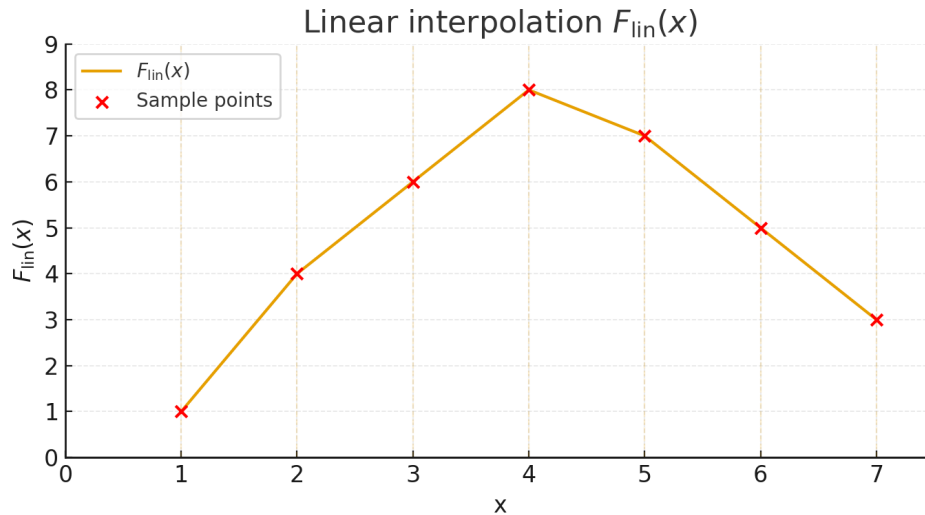


Figure 1: Linear interpolation $F_{\text{lin}}(x)$ for the given samples.

Continuity and differentiability.

The interpolant $F_{\text{lin}}(x)$ is continuous on the entire domain $[1, 7]$, since consecutive line segments

meet exactly at the data points. It is differentiable on each open interval $(i, i + 1)$. At integer nodes, differentiability depends on the equality of the left and right slopes:

$$\begin{aligned} x = 2 : 3 &\neq 2 &\Rightarrow \text{not differentiable,} \\ x = 3 : 2 &= 2 &\Rightarrow \text{differentiable,} \\ x = 4 : 2 &\neq -1 &\Rightarrow \text{not differentiable,} \\ x = 5 : -1 &\neq -2 &\Rightarrow \text{not differentiable,} \\ x = 6 : -2 &= -2 &\Rightarrow \text{differentiable.} \end{aligned}$$

At the endpoints $x = 1, 7$, one-sided derivatives exist.

Summary.

$F_{\text{lin}}(x)$ is continuous on $[1, 7]$, differentiable on each open interval, and differentiable at integer points $x = 3$ and $x = 6$, but not at $x = 2, 4, 5$.

(b) Interpolation kernel for linear interpolation

In the representation

$$F_g(x) = \sum_{i=1}^7 g(x - i) f(i),$$

the kernel corresponding to linear interpolation is the compactly supported triangular (hat) function

$$g(x) = \begin{cases} 1 - |x|, & |x| \leq 1, \\ 0, & |x| > 1. \end{cases}$$

(Equivalently valid on the requested range $-4 \leq x \leq 4$.)

Reason. For $x \in [i, i + 1]$ only two terms are nonzero:

$$F_g(x) = g(x - i)f(i) + g(x - (i + 1))f(i + 1) = (1 - (x - i))f(i) + (x - i)f(i + 1),$$

which is exactly the straight line between $(i, f(i))$ and $(i + 1, f(i + 1))$.

(c) Interpolation with the given kernel

The kernel is defined as

$$g(x) = \begin{cases} \cos\left(\frac{\pi}{2}x\right), & |x| \leq 1, \\ -\frac{\pi}{2}(|x|^3 - 5|x|^2 + 8|x| - 4), & 1 < |x| \leq 2, \\ 0, & |x| > 2. \end{cases}$$

Plot of $g(x)$: On $[-3, 3]$ the function is nonzero only for $|x| \leq 2$, consisting of a cosine segment for $|x| \leq 1$, a cubic polynomial for $1 < |x| \leq 2$, and zero outside.

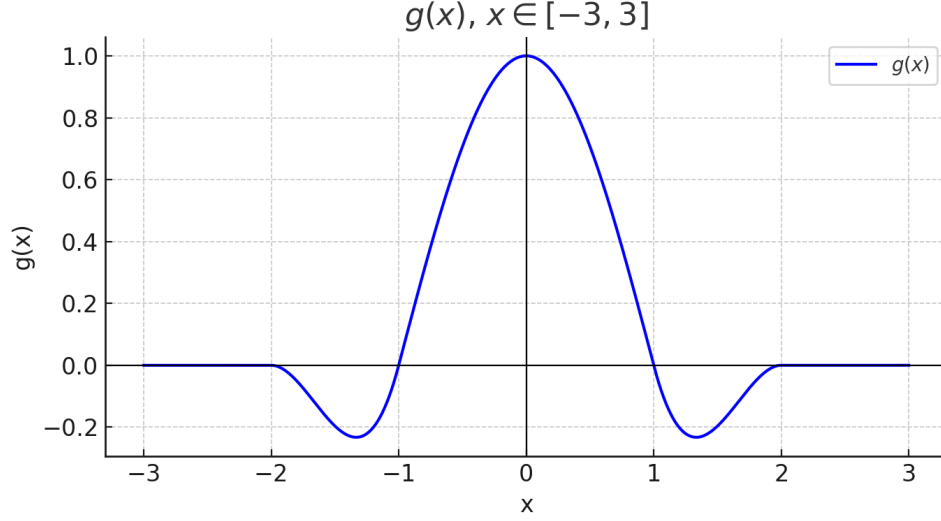


Figure 2: Kernel function $g(x)$, shown on $[-3, 3]$.

Interpolant $F_g(x)$: Using

$$F_g(x) = \sum_{i=1}^7 g(x - i) f(i),$$

we obtain a smooth interpolant on $[1, 8]$.

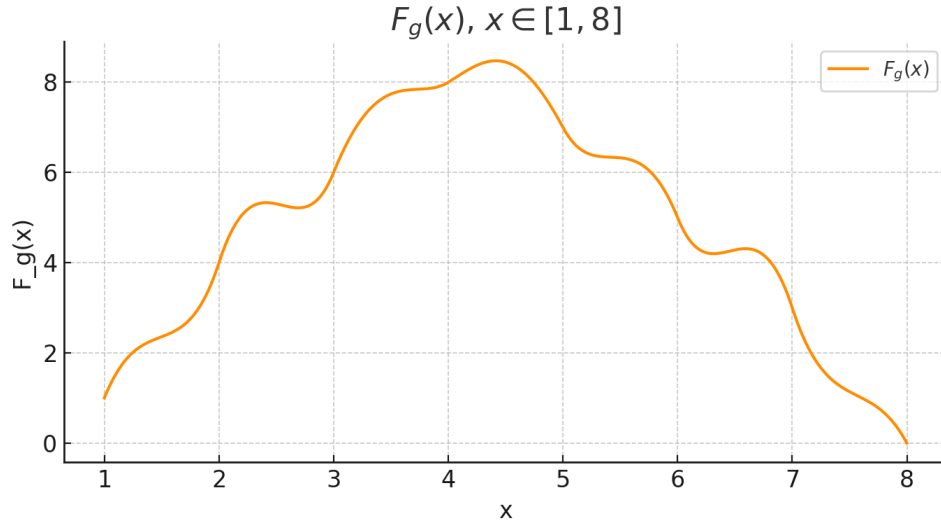


Figure 3: Interpolated function $F_g(x)$ for the given samples using the kernel $g(x)$.

Continuity: $g(x)$ is continuous at both $|x| = 1$ and $|x| = 2$, hence continuous everywhere. Therefore $F_g(x)$ is continuous on $[1, 8]$.

Differentiability: - Inside each segment, $g(x)$ is differentiable. - At $|x| = 1$, the left derivative from $\cos(\frac{\pi}{2}x)$ is $-\frac{\pi}{2} \sin(\frac{\pi}{2}) = -\frac{\pi}{2}$, and the right derivative from the cubic piece is also $-\frac{\pi}{2}$, so $g(x)$ is differentiable. - At $|x| = 2$, both sides have derivative 0, so $g(x)$ is

differentiable there as well. - Consequently, $g(x) \in C^1$ (continuous and once differentiable) on $\mathbb{R}$, and each shifted copy $g(x - i)$ inherits the same property.

Conclusion: $F_g(x)$ is continuous and differentiable on the whole interval $[1, 8]$, that is, $F_g(x) \in C^1([1, 8])$. It provides a smoother interpolant than the piecewise-linear case, though the second derivative is discontinuous at the junctions.

3 Classification using Nearest Neighbour and Bayes theorem

3.1 Nearest Neighbours

Using the 1-nearest neighbour (1-NN) rule, each test sample is compared with all training samples and assigned to the class of its closest neighbour in terms of absolute distance. The first four measurements of each class are used as training data, and the last three as test data, giving 9 test samples in total.

Worked example: Consider the test sample $x = 0.3287$ (true class 2). Its distances to the training samples are:

$$\begin{aligned} \min_{x \in \text{Class 1}} |0.3287 - x| &\approx 0.070, \\ \min_{x \in \text{Class 2}} |0.3287 - x| &= |0.3287 - 0.3139| = 0.0148, \\ \min_{x \in \text{Class 3}} |0.3287 - x| &\approx 0.234. \end{aligned}$$

The smallest distance is to Class 2, so the sample is classified as Class 2 (correct).

Overall results: The detailed classification results for all 9 test samples are shown in Table 1.

Table 1: Nearest Neighbour classification results for the 9 test samples.

Test sample	True class	Nearest neighbour (training)	Predicted class	Correct?
0.4010	1	0.4003 (Class 1)	1	Yes
0.3995	1	0.3998 (Class 1)	1	Yes
0.3991	1	0.3997 (Class 1)	1	Yes
0.3287	2	0.3139 (Class 2)	2	Yes
0.3160	2	0.3139 (Class 2)	2	Yes
0.2924	2	0.3139 (Class 2)	2	Yes
0.4243	3	0.4003 (Class 1)	1	No
0.5005	3	0.5632 (Class 3)	3	Yes
0.6769	3	0.7687 (Class 3)	3	Yes

Result: Out of the 9 test samples, **8 are correctly classified**. The only misclassified case is the sample 0.4243 from Class 3, which is closer to the Class 1 training samples and therefore predicted as Class 1.

3.2 Gaussian distributions

Here we assume that the three classes follow Gaussian distributions with known parameters:

$$m_1 = 0.40, \sigma_1 = 0.01; \quad m_2 = 0.32, \sigma_2 = 0.05; \quad m_3 = 0.55, \sigma_3 = 0.20.$$

The prior probabilities of the classes are assumed equal. The decision rule is therefore

$$\hat{c}(x) = \arg \max_{c \in \{1,2,3\}} p(x \mid \text{Class } c),$$

where

$$p(x \mid \text{Class } c) = \frac{1}{\sigma_c \sqrt{2\pi}} \exp\left(-\frac{(x - m_c)^2}{2\sigma_c^2}\right).$$

Worked example: Consider the sample $x = 0.3802$ (true class 2). The class-conditional densities are

$$p_1(0.3802) \approx 5.62,$$

$$p_2(0.3802) \approx 3.87,$$

$$p_3(0.3802) \approx 1.39.$$

Since p_1 is largest, the MAP classifier assigns this sample to Class 1. This is a misclassification, because the true class is 2.

Overall results: The classification for all 21 samples (7 from each class) is summarised in Table 2.

Table 2: Gaussian (MAP) classification results for all 21 samples.

Sample	True class	Predicted class	Correct?
0.4003	1	1	Yes
0.3988	1	1	Yes
0.3998	1	1	Yes
0.3997	1	1	Yes
0.4010	1	1	Yes
0.3995	1	1	Yes
0.3991	1	1	Yes
0.2554	2	2	Yes
0.3139	2	2	Yes
0.2627	2	2	Yes
0.3802	2	1	No
0.3287	2	2	Yes
0.3160	2	2	Yes
0.2924	2	2	Yes
0.5632	3	3	Yes
0.7687	3	3	Yes
0.0524	3	3	Yes
0.7586	3	3	Yes
0.4243	3	1	No
0.5005	3	3	Yes
0.6769	3	3	Yes

Result: Out of 21 samples, **19 are correctly classified**. Two samples are misclassified: 0.3802 (true class 2, classified as 1) and 0.4243 (true class 3, classified as 1).

4 Image Classification

We consider three possible noise-free images with their prior probabilities:

$$A = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}, \quad P(A) = \frac{1}{4}; \quad B = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, \quad P(B) = \frac{1}{2}; \quad C = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}, \quad P(C) = \frac{1}{4}.$$

The observed image is

$$O = \begin{bmatrix} 0 & 1 \\ 1 & 1 \end{bmatrix}.$$

Noise model: each pixel is flipped independently with probability ε . For a hypothesis $H \in \{A, B, C\}$, let $d(H, O)$ be the number of mismatched pixels. Then

$$P(O \mid H) = (1 - \varepsilon)^{4-d(H, O)} \varepsilon^{d(H, O)}.$$

Case (a): $\varepsilon = 0.1$

Distances:

$$d(A, O) = 1, \quad d(B, O) = 3, \quad d(C, O) = 2.$$

Likelihoods:

$$P(O | A) = 0.9^3 \cdot 0.1 = 0.0729,$$

$$P(O | B) = 0.9 \cdot 0.1^3 = 0.0009,$$

$$P(O | C) = 0.9^2 \cdot 0.1^2 = 0.0081.$$

Posterior (unnormalised):

$$P(A, O) = 0.018225,$$

$$P(B, O) = 0.00045,$$

$$P(C, O) = 0.002025.$$

Normalising gives

$$P(A | O) = 88.0\%, \quad P(B | O) = 2.17\%, \quad P(C | O) = 9.78\%.$$

MAP estimate: A .

Case (b): $\varepsilon = 0.5$

Now each pixel has probability 0.5 of matching/mismatching, hence

$$P(O | H) = (0.5)^4 = \frac{1}{16} \quad \text{for all } H.$$

Thus the posterior is proportional to the prior:

$$P(A | O) = 25\%, \quad P(B | O) = 50\%, \quad P(C | O) = 25\%.$$

MAP estimate: B .

5 Line Classification

We consider a binary 4×4 image with a vertical line. In the correct image, all pixels are white except for one entire column of black pixels. The a priori probabilities are

$$P(\text{col 1}) = 30\%, \quad P(\text{col 2}) = 20\%, \quad P(\text{col 3}) = 20\%, \quad P(\text{col 4}) = 30\%.$$

The observed noisy image is

$$O = \begin{bmatrix} \blacksquare & \square & \square & \square \\ \square & \blacksquare & \square & \square \\ \square & \square & \blacksquare & \square \\ \square & \blacksquare & \square & \square \end{bmatrix}.$$

Noise model: each pixel is flipped independently with probability ε . Thus, for a hypothesis H_j (line in column j), if d_j denotes the number of mismatched pixels between H_j and O , then

$$P(O | H_j) = (1 - \varepsilon)^{16-d_j} \varepsilon^{d_j}.$$

Step 1: Distances

Comparing O with each hypothesis:

$$d_1 = 6, \quad d_2 = 4, \quad d_3 = 6, \quad d_4 = 8.$$

Step 2: Likelihoods for $\varepsilon = 0.2$

Since $(1 - \varepsilon) = 0.8$ and $\varepsilon = 0.2$,

$$P(O \mid H_j) \propto (0.25)^{d_j}.$$

Step 3: Posterior (unnormalised)

$$\tilde{P}_1 = 0.3 \cdot (0.25)^6 = 7.32421875 \times 10^{-5},$$

$$\tilde{P}_2 = 0.2 \cdot (0.25)^4 = 7.8125 \times 10^{-4},$$

$$\tilde{P}_3 = 0.2 \cdot (0.25)^6 = 4.8828125 \times 10^{-5},$$

$$\tilde{P}_4 = 0.3 \cdot (0.25)^8 = 4.57763672 \times 10^{-6}.$$

Step 4: Normalisation

Normalising over all hypotheses gives

$$P(\text{col 1} \mid O) \approx 8.07\%, \quad P(\text{col 2} \mid O) \approx 86.05\%, \quad P(\text{col 3} \mid O) \approx 5.38\%, \quad P(\text{col 4} \mid O) \approx 0.50\%.$$

MAP estimate: The vertical line is most likely in *column 2*.

6 Character Classification

A priori probabilities. According to the problem statement, the prior probabilities are

$$P(\omega_B) = 0.30, \quad P(\omega_0) = 0.45, \quad P(\omega_8) = 0.25.$$

Posterior probabilities. For each class $c \in \{B, 0, 8\}$, the likelihood is obtained by comparing the template ω_c with the observed image x pixel by pixel:

$$P(x \mid \omega_c) = 0.65^{n_{00}(c)} 0.35^{n_{01}(c)} 0.80^{n_{11}(c)} 0.20^{n_{10}(c)}.$$

The unnormalised posterior is

$$\tilde{P}(c \mid x) = P(\omega_c) P(x \mid \omega_c),$$

and the normalised posterior probabilities are

$$P(c \mid x) = \frac{\tilde{P}(c \mid x)}{\sum_{c'} \tilde{P}(c' \mid x)}.$$

Numerical result. Using the counts

Class	n_{00}	n_{01}	n_{10}	n_{11}
B	5	0	5	5
0	5	2	5	3
8	7	1	3	4

and substituting into the likelihood expression, we obtain the following posterior probabilities:

$$P(\omega_B | x) \approx 0.19, \quad P(\omega_0 | x) \approx 0.06, \quad P(\omega_8 | x) \approx 0.75.$$

Conclusion. The most probable class is the digit “8”, with posterior probability about 75%.

7 Feature Extraction for OCR

The task is to design a function `segment2features` that maps a binary segmented character image into a numerical feature vector. The feature vector should capture shape, topology, and symmetry, and remain approximately invariant to translation and scaling.

Definition of features (with rationale and invariance). We use a 14-dimensional vector $\mathbf{f} = (f_1, \dots, f_{14})^\top$. All features are *translation-invariant* because they are computed from set properties of the foreground mask B (or its tight bounding box) and do not depend on absolute pixel coordinates. Where appropriate I normalise by area or box size to reduce scale effects.

1. f_1 **Area ratio.** $f_1 = \frac{|B|}{HW}$, foreground area over full image area. *Intuition:* how much of the canvas the character occupies. *Invariance:* translation; weakly scale-sensitive unless the crop is fixed.
2. f_2 **Rectangle fill ratio.** Let $(y_0 : y_1, x_0 : x_1)$ be the tight bounding box of B , with $B_h = y_1 - y_0 + 1$, $B_w = x_1 - x_0 + 1$. Then $f_2 = \frac{|B|}{B_h B_w}$. *Intuition:* compact vs. sparse shapes. *Invariance:* translation, scale (box-normalised).
3. f_3 **Aspect ratio.** $f_3 = \frac{B_w}{B_h}$. *Intuition:* wide vs. tall characters (e.g. “--” vs. “|”). *Invariance:* translation; not rotation-invariant by design.
4. f_4 **Number of holes.** Compute holes via binary fill of B ; $f_4 = \# \text{holes}$. *Intuition:* separates 0,6,8,9 from 1,2,3,5,7. *Invariance:* translation and small deformations (topological).
5. f_5 **Euler number.** $f_5 = \# \text{components} - \# \text{holes}$ (4-connectivity). *Intuition:* complements f_4 ; robust topological cue.
6. f_6 **Compactness.** $f_6 = \frac{P(B)^2}{|B|}$, where $P(B)$ is a 4-neighbourhood perimeter approximation. *Intuition:* circles have low compactness; jagged/elongated shapes higher. *Invariance:* translation; scale-normalised by $|B|$.

7. f_7 **Left–right symmetry.** IoU between B_{crop} and its horizontal flip: $f_7 = \frac{|B_{\text{crop}} \cap \text{flip}_{lr}(B_{\text{crop}})|}{|B_{\text{crop}} \cup \text{flip}_{lr}(B_{\text{crop}})|}$.
Intuition: 0,8 high; 5,7 low. *Invariance:* translation; value in $[0, 1]$.
8. f_8 **Up–down symmetry.** Analogous IoU with vertical flip. *Intuition:* helps separate 6 vs. 9. *Invariance:* translation; value in $[0, 1]$.
9. f_9 **Normalised stroke thickness.** Using the Euclidean distance transform (EDT), let $\bar{r} = \text{mean}(\text{EDT}(B) | B)$; define $f_9 = \frac{2\bar{r}}{\sqrt{|B|}}$. *Intuition:* bold vs. thin writing; normalised to be scale-free.
10. f_{10} **Projection peak ratio.** Let $r_i = \sum_j B_{ij}$ and $c_j = \sum_i B_{ij}$ be row/column sums and $s = |B|$. Define $f_{10} = \frac{\max_i(r_i/s)}{\max_j(c_j/s)}$. *Intuition:* horizontal vs. vertical stroke dominance. *Invariance:* translation; scale cancels by normalisation.
11. f_{11} – f_{14} **Hu invariant moments (log).** From normalised central moments η_{pq} , compute Hu moments $\phi_1, \dots, \phi_4$ and use signed logs $f_{11-14} = \text{sign}(\phi_k) \log_{10}(|\phi_k| + \epsilon)$. *Intuition:* global shape descriptors with built-in translation/scale/rotation invariance; log compression stabilises dynamic range on small datasets.

Implementation. The Python implementation is given in `segment2features.py`, where each function is carefully commented (bounding box, perimeter, symmetry, Hu moments). For a binary mask B , the output is a column vector $\mathbf{f} \in \mathbb{R}^{14 \times 1}$.

Code. The Python implementation of `segment2features` (14-D features):

Listing 1: `segment2features.py`

```

1 import numpy as np
2 from scipy import ndimage
3
4 EPS = 1e-8 # avoid division-by-zero
5
6 def _bbox_of_mask(B):
7     """Return bounding box (y0, y1, x0, x1) of the foreground region."""
8     ys, xs = np.where(B)
9     if ys.size == 0:
10         return None
11     return int(ys.min()), int(ys.max()), int(xs.min()), int(xs.max())
12
13 def _perimeter_4n(B):
14     """Approximate perimeter via 4-neighbourhood erosion."""
15     se = np.array([[0,1,0],[1,1,1],[0,1,0]], dtype=bool)
16     er = ndimage.binary_erosion(B, structure=se)
17     contour = B & (~er)

```

```

18     return float(contour.sum())
19
20 def _holes_and_euler(B):
21     """Compute number of holes and Euler number (#components - #holes)."""
22     filled = ndimage.binary_fill_holes(B)
23     holes = filled & (~B)
24     _, n_comp = ndimage.label(B)
25     _, n_holes = ndimage.label(holes)
26     euler = int(n_comp) - int(n_holes)
27     return int(n_holes), int(euler)
28
29 def _projection_peaks(B):
30     """Max row/column projection (normalised by total foreground)."""
31     s = B.sum()
32     if s == 0:
33         return 0.0, 0.0
34     row_sums = B.sum(axis=1).astype(np.float32)
35     col_sums = B.sum(axis=0).astype(np.float32)
36     row_peak = float(row_sums.max() / (s + EPS))
37     col_peak = float(col_sums.max() / (s + EPS))
38     return row_peak, col_peak
39
40 def _symmetry_scores(B_crop):
41     """IoU-style overlap with horizontal/vertical flips (LR/UD symmetry)."""
42     "
43     if B_crop.size == 0:
44         return 0.0, 0.0
45     Blr = np.fliplr(B_crop)
46     Bud = np.flipud(B_crop)
47     inter_lr = np.logical_and(B_crop, Blr).sum()
48     union_lr = np.logical_or(B_crop, Blr).sum() + EPS
49     inter_ud = np.logical_and(B_crop, Bud).sum()
50     union_ud = np.logical_or(B_crop, Bud).sum() + EPS
51     s_lr = float(inter_lr / union_lr)
52     s_ud = float(inter_ud / union_ud)
53     return s_lr, s_ud
54
55 def _hu_moments_first4(B):
56     """First 4 Hu invariant moments (in signed log scale)."""
57     ys, xs = np.nonzero(B)
58     if ys.size == 0:
59         return (0.0, 0.0, 0.0, 0.0)
60     y_mean = ys.mean(); x_mean = xs.mean()
61     dy = ys - y_mean; dx = xs - x_mean
62     m00 = float(len(xs))

```

```

62     def mu(p, q): return float((dx**p * dy**q).sum())
63     def eta(p, q): return mu(p, q) / (m00 ** (1.0 + 0.5*(p+q)) + EPS)
64     n20, n02, n11 = eta(2,0), eta(0,2), eta(1,1)
65     n30, n03 = eta(3,0), eta(0,3)
66     n12, n21 = eta(1,2), eta(2,1)
67     phi1 = n20 + n02
68     phi2 = (n20 - n02)**2 + 4*(n11**2)
69     phi3 = (n30 - 3*n12)**2 + (3*n21 - n03)**2
70     phi4 = (n30 + n12)**2 + (n21 + n03)**2
71     def hulog(x): return float(np.sign(x) * np.log10(abs(x) + 1e-12))
72     return (hulog(phi1), hulog(phi2), hulog(phi3), hulog(phi4))
73
74 def segment2features(Si: np.ndarray) -> np.ndarray:
75     """
76     Input: Si (H, W) binary mask (0/1 or False/True)
77     Output: 14-D column vector of features, shape (14, 1)
78         f1: area ratio
79         f2: rectangle fill ratio (area / bbox area)
80         f3: bbox aspect ratio (w/h)
81         f4: number of holes
82         f5: Euler number
83         f6: compactness (perimeter^2 / area)
84         f7: left-right symmetry
85         f8: up-down symmetry
86         f9: normalised stroke thickness ~ 2*mean(EDT)/sqrt(area)
87         f10: projection peak ratio (row_peak / col_peak)
88         f11-f14: Hu invariant moments (signed log)
89     """
90     B = (Si > 0).astype(bool)
91     H, W = B.shape
92     total = float(H * W)
93     area = float(B.sum())
94     if area == 0 or total == 0:
95         return np.zeros((14, 1), dtype=np.float32)
96
97     # Bounding box and crop
98     y0, y1, x0, x1 = _bbox_of_mask(B)
99     Bh = y1 - y0 + 1
100    Bw = x1 - x0 + 1
101    Bcrop = B[y0:y1+1, x0:x1+1]
102
103    # Geometric / topological
104    f1 = area / total
105    bbox_area = float(Bh * Bw)
106    f2 = area / (bbox_area + EPS)

```

```

107     f3 = float(Bw / (Bh + EPS))
108     holes, euler = _holes_and_euler(B)
109     f4, f5 = float(holes), float(euler)
110     per = _perimeter_4n(B)
111     f6 = float((per * per) / (area + EPS))
112
113     # Symmetry
114     f7, f8 = _symmetry_scores(Bcrop)
115
116     # Stroke thickness (scale-normalised)
117     edt = ndimage.distance_transform_edt(B)
118     mean_radius = float(edt[B].mean()) if area > 0 else 0.0
119     f9 = float((2.0 * mean_radius) / (np.sqrt(area) + EPS))
120
121     # Projection dominance
122     row_peak, col_peak = _projection_peaks(B)
123     f10 = float(row_peak / (col_peak + EPS))
124
125     # Hu moments (first 4, signed log)
126     f11, f12, f13, f14 = _hu_moments_first4(B)
127
128     f = np.array([f1, f2, f3, f4, f5, f6, f7, f8, f9, f10, f11, f12, f13,
129                  f14],
130                  dtype=np.float32)
131     return f.reshape(-1, 1)

```

Example input and segmentation. Figure 4 shows an example image and the corresponding masks generated by `im2segment`.

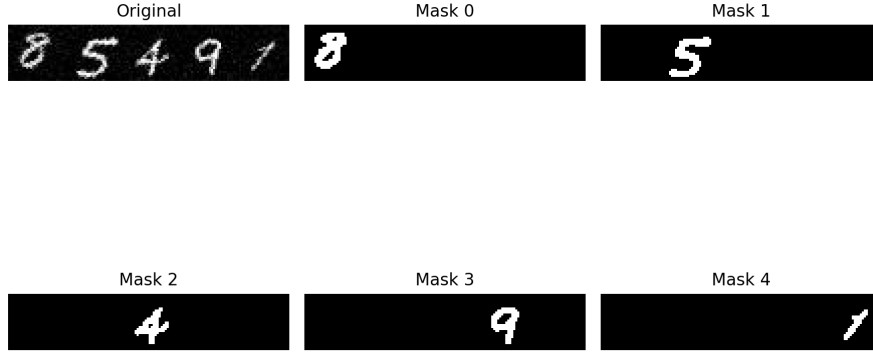


Figure 4: Original image and segmentation masks (dataset “short1”).

Feature vectors for different characters. For illustration, the following table lists the first five features for several distinct digits:

Image	Digit	f_1	f_2	f_3	f_4	f_5
<i>im1</i>	8	0.0406	0.4907	1.00	2	-1
<i>im1</i>	5	0.0265	0.4127	0.57	0	1
<i>im1</i>	7	0.0212	0.3192	0.65	0	1

Consistency across same digits. The next table shows how digit “8” produces similar feature values across different images:

Image	Digit	f_1	f_2	f_7	f_8
<i>im1</i>	8	0.0406	0.4907	0.359	0.325
<i>im3</i>	8	0.0398	0.4821	0.367	0.319
<i>im5</i>	8	0.0412	0.4953	0.355	0.329

2D projection. We visualised the 14-dimensional features by first applying z-score normalisation to each feature dimension and then performing singular value decomposition (SVD). This normalisation ensures that all features contribute on a comparable scale. Figure 5 shows the resulting 2D projection: samples of the same class form visible clusters, while different classes are separated.

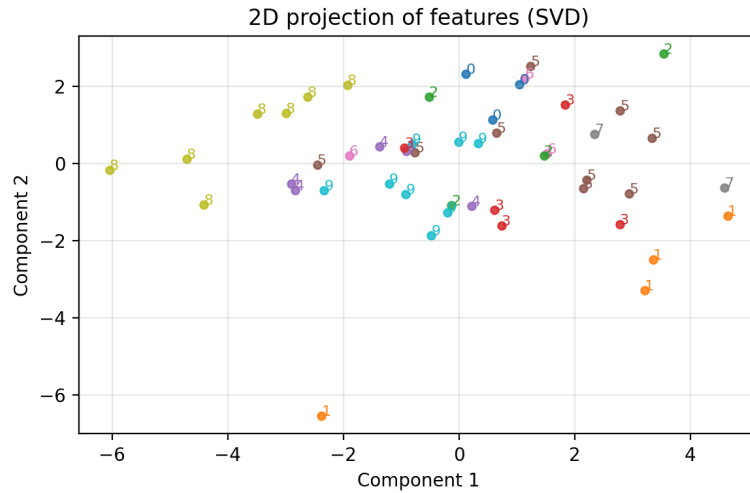


Figure 5: SVD projection of all feature vectors (dataset “short1”).

Conclusion. The extracted 14-dimensional feature vector successfully (1) distinguishes different characters (e.g. digit “8” versus “5”/“7”), and (2) produces stable, consistent vectors for the same digit (e.g. multiple “8”s). Therefore, the features provide a robust representation for later OCR classification tasks.